IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

A Multi-source Log Hidden Anomaly Detection Method Integrating Trans-Encoder and LSTM

Chen Luo[1], Hongcheng Jiang[2], Peng Zhou[2], Xinzhe Wang[1], Zhipeng Shao[1], and Wangshu Sun[2]

[1] China Electric Power Research Institute, State Grid Laboratory of Power Cyber-Security Protection and Monitoring Technology, Nanjing 210003, Jiangsu, China
[2] State Grid Zhejiang Electric Power CO., Ltd., Hangzhou, Zhejiang 310007, China

Corresponding author: Chen Luo (e-mail: luochenll1@163.com).

This work is supported by the science and technology project of State Grid Corporation of China: "Defense System Design and Key Technologies Research on Cyber Security of New Power System" (Grand No. 5700-202358388A-2-3-XG)

ABSTRACT Identifying hidden anomalous behavior is a major challenge in anomaly detection, particularly in complex systems where anomalies are buried within massive log data and cannot be easily identified through simple patterns or rules. To address this issue, we propose a novel approach that integrates a Transformer encoder module with Long Short-Term Memory (LSTM) to detect covert anomalies in logs using only normal class datasets for training. First, we enhance feature extraction from multi-source log data by improving the Transformer encoder structure with an added masking mechanism. Then, we apply LSTM for time-series modeling to capture temporal correlations between extracted features, improving the model's ability to analyze sequential dependencies. Finally, we evaluate the semantic and temporal consistency of operations to refine anomaly detection. Experimental results demonstrate that our approach effectively detects hidden anomalies, achieving improvements of 2.3% in precision, 4.9% in recall, and 3.4% in F1 score. Moreover, it maintains stable performance with just 10% of the training data and improves computational efficiency in the test phase by 16.98% compared to DeepLog, highlighting its potential for practical application in anomaly detection.

INDEX TERMS Hidden Anomalies, Data Security, Unsupervised Learning, Anomaly Detection.

I. INTRODUCTION

Against the backdrop of digital transformation, data security is facing serious challenges. Among them, insider threats have become one of the leading causes of data breaches. The Ponemon Institute's 2023 report[1] shows that the average global cost of an insider threat is as high as $11.45 million, underscoring the magnitude of the problem. However, existing anomaly detection methods have significant shortcomings in identifying hidden anomalous behaviors, which stems from two key challenges: firstly, anomalous events tend to arise from concurrent user operations rather than isolated events; and secondly, existing methods tend to use one-dimensional log analysis, which makes it difficult to effectively capture complex behavioral patterns.

Deep learning techniques have made breakthroughs in the critical area of hidden anomaly detection. Deep learning techniques have made significant progress in the field of covert anomaly detection. Literature[2] proposed a GRU-based insider threat detection method, which effectively captures the historical and current information on user behavior by converting user behavior data into numerical vectors and feeding them into a GRU network. Liu et al[3], on the other hand, developed an anomaly detection system based on the integration of a self-encoder, which identifies potential insider threats using the reconstruction error. However, there is a general problem with the existing methods: there exists no consideration of log information having semantic information when processing the data. However, it is undeniable that these studies provide important references for us, and there are the following research gaps: (1) existing methods fail to adequately integrate the semantic features and time series correlation of log data; (2) cannot detect hidden anomalies in concurrent operations; and (3) the generalization performance of the existing models needs to be improved.

Based on the above research status and challenges, this paper proposes a covert anomaly detection method based on TE-LSTM. The method innovatively combines the advantages of the Transformer encoder and LSTM network and effectively improves the detection performance of the model in complex scenarios by improving the encoder module and introducing the masking mechanism. The contribution of this paper is as follows:

1. The semantic features and time series correlation of the log data are fully taken into account, allowing the encoder and LSTM to play out their respective advantages and improve the accuracy of anomaly detection.

2. The Transformer-based encoder module is improved by removing the positional encoding and increasing the masking mechanism so that TE-LSTM can be better adapted to the application scenarios in this paper.

3. The investigation and demonstration of the ability of the proposed model to detect hidden anomalies in log data. The effectiveness of the model in detecting hidden anomalies will be fully verified.

The rest of the paper is structured as follows. Section 2 discusses existing methods for unsupervised detection of hidden anomalies. Section 3 defines the problem discussed in this paper and the details of the TE-LSTM model. Section 4 describes the details of the implementation method. Section 5 describes the experiments showing that TE-LSTM has a better performance in detecting hidden anomalies. Section 6 concludes this paper and plans for future work.

II. Related Work

We achieve anomaly detection by combining contextual features and temporal associations, so this section describes the current approach of feature vector extraction and the current state of research in anomaly detection.

A. Feature Vector Extraction

Multi-source log aggregation performs data fusion, which is generally divided into three levels: data layer fusion, feature layer fusion, and decision layer fusion[4]. Feature-based fusion is in the process of the feature layer, and data fusion is performed in the intermediate layer of data processing. The original feature-based fusion method is relatively crude, which directly concatenates features from multiple sources of heterogeneous information to form a new feature vector, which is then used for clustering or classification analysis. This feature fusion method ignores the redundancy between the features of multi-source heterogeneous information, and the relevance and effect are not satisfactory. The emergence of the deep learning method improves the features of multi-source heterogeneous data and makes great progress in the feature fusion effect, which makes the research of multi-source heterogeneous data fusion a big step forward.

Eldardiry et al.[5] propose a method for merging different types of data. The method defines the consistency between different types of data in terms of the user's workgroup attributes and then tests the consistency of the user's different types of data. The method relies on the assumption of consistency of the user's group attributes, which is not in line with the general practical situation. Zhang et al.[6] combined the data mining algorithm FP-Growth to use each second as the time interval for log splitting. Logs generated in the same time interval are grouped into that time interval, and then all transaction items in the same time interval are merged to form the structure of the transaction database. Huang et al.[7] proposed the HitAnomaly system, which models log template sequences and parameter values using a hierarchical transformer structure. They designed a log sequence encoder and parameter value encoder and used an attention mechanism as a classification model to handle various types of anomalies by capturing semantic information from log template sequences and parameter values. Yuan et al.[8] used the LSTM model to capture the sequential information of user action sequences. When constructing user behavior representations, they introduced an attention layer to allow the model to focus on the behavior of individual users. Experimental results show that the model successfully identifies insider threats and outperforms other deep learning models. The LSTM model itself has a strong dependence on long sequences, and there may be insufficient information fusion when dealing with multi-source data. Furthermore, although the attention layer is introduced to enhance the focus on individual user behavior, it may not effectively address the interactions between different behavioral features, which affects the model's performance in complex environments.

There are fewer studies on insider threat detection through fusion features. Existing research using fusion features has made some progress in the area of CV, which provides ideas for studying the fusion of insider threat data from multiple sources. Lee et al.[9] exploited the property that residual connectivity allows low-level information to be retained as the network propagates. They extended the core idea of residual learning to semantic image segmentation. High-resolution prediction was achieved by effectively learning combinations of fused features through multi-level feature refinement blocks. Similarly, Bilinski et al.[10] incorporated shortcuts to capture contextual information in the decoder network. They allowed information to propagate efficiently from one decoder block to another, enabling multi-level feature fusion, which significantly improves the accuracy of single-channel semantic segmentation.

Data mining techniques are an innovative application in log feature fusion, but the results of extracting feature vectors are unsatisfactory due to concurrent user operations, simultaneous recording of multiple transactions, and the appearance of multiple feature values, which lead to a large number of frequent item sets. By combining the strengths and weaknesses of the above methods, a deep learning approach for multi-source data fusion is employed to extract semantic

feature vectors from logs, thereby improving the accuracy of anomaly detection.

B. Anomaly Detection

Le et al.[11] demonstrated that parsed logs may suffer from OOV errors or semantic misinterpretation in subsequent anomaly detection. Therefore, they proposed a novel log-based anomaly detection method called NeuralLog, which does not require log parsing but directly extracts the semantics of the raw log messages and represents them as semantic vectors. The vectors are then used to detect anomalies via a transformer-based classification model. Liu[12] argued that an effective method for anomaly detection is to extract key representations from normal samples. Based on this idea, they proposed a semi-supervised method called LCR-GAN, which can only reconstruct the normal samples but not the abnormal samples, thus achieving excellent anomaly detection performance by obtaining lower reconstruction error for normal samples and higher reconstruction error for anomalous samples.

Due to the large volume and high dimensionality of time series data, its anomaly detection faces complexity problems. Mao et al.[13] proposed an unsupervised anomaly detection method for time series data based on Generative Adversarial Networks (GANs). The method uses Long Short-Term Memory Recurrent Neural Networks (LSTM-RNN) as the basic module. The generative model is used to capture the distribution of high-dimensional data and the LSTM module is used to learn the temporal relationships. The proposed framework can reconstruct the input data after training and assign anomaly scores to each time step in the time series based on the reconstruction error to effectively identify anomalies. Said Elsayed et al.[14] argued that an important issue in anomaly detection techniques is the availability of labeled data for model training and validation, and therefore proposed an ultra-methodology based on Long Short-Term Memory (LSTM) auto-encoders and One-Class Support Vector Machines (OC-SVMs). The LSTM auto-encoder is trained to learn the normal traffic patterns and to learn compressed representations (i.e., latent features) of the input data to detect attack-based anomalies in an unbalanced dataset by training the model using examples from the normal class.

The idea of anomaly detection in the above methods is to detect anomalies by learning normal behavioral sequences. However, these methods only analyze semantics to detect contextual anomalies or focus solely on temporal relationships to detect anomalous sequences. Log contextual anomaly detection can capture complex dependencies and patterns between log entries. For example, an anomalous event may not occur in isolation but is related to a sequence or combination of other events, which can reveal some hidden anomalous patterns. On the other hand, considering the temporal correlation between sequences of log transactions captures information related to more distant time steps in the data sequence. An effective approach is to train LSTM models. However, as mentioned in the previous section, the concurrent execution of user operations poses a challenge in analyzing the temporal relationships. Therefore, relying only on semantic or temporal relationships does not apply to detecting hidden anomalies in multi-source logs.

Therefore, we propose an anomaly detection method that preserves the semantic features and temporal relationships of the logs and learns normal operation sequences for anomaly detection. The method performs multi-source log feature fusion by enhancing the Transformer-Encoder module to extract the semantic features from the logs and analyzes the temporal features of the sequences in conjunction with LSTM.

III. METHOD

We designed a Transformer-LSTM hybrid anomaly detection method, the method architecture is shown in Fig. 1, the method has two working phases: offline training phase and online detection phase. In the training phase, the data is first processed by preprocessing. The main task of preprocessing is to remove noisy operation sequences by log parsing, extract each feature value in the separated sequences to improve the data quality, and prepare for training the model. The anomaly detection module uses the data after preprocessing to train the TE-LSTM (Trans-Encoder and LSTM log anomaly detection method) model to learn the semantics and temporal associations of normal sequences. Trans-Encoder can capture the global semantic dependencies in log sequences, while the LSTM component excels at learning long-term temporal dependencies, ensuring that the model can effectively distinguish normal patterns from anomalous behaviors. In the detection phase, known (or already flagged) anomalous behavior is first directly filtered out. Next, the trained TE-LSTM is used to evaluate whether the current operation matches the preceding series of operations in both semantic and temporal dimensions. Since the TE-LSTM learns only normal sequences, it can detect unknown hidden anomalous behavior and mark it as an anomaly. Furthermore, the detected anomalous behavior can be sent to a domain expert for further investigation and action, such as banning the user or even revalidating the system software.

This paper begins with a formal presentation of the problem to be addressed in this paper in Section 3.1. The composition of the TE-LSTM model is described in detail in Section 3.2.

A. Problem formulation

The log can now be represented by the following equation, if each entry in the multi-source log is defined as a transaction:

$$Z = \{T_1, T_2, \ldots\ldots, T_N\}, \quad (1)$$

Where Z is the set of transactions, i.e. the original time series, T_i is the i transaction in Z, and N is the total number of things in the log. For $T_i \in Z$, where $1 < i < N$, this paper describes it by equation (2):

$$T_i = \{x_1, x_2, \ldots\ldots, x_n\}, \quad (2)$$

Where $x_i \in R^m$, $1 < i < n$ is each feature field of T_i, specifically x_i represents the information of a transaction

record, e.g. timestamp, date, level, etc. It is now compressed into a feature vector containing n features, i.e. $\widehat{T_i}$. As in equation (3), $\hat{Z}$ is the feature space after extracting the features from the log, and a unique $T_i \in Z$ can be found for each $\widehat{T_i} \in \hat{Z}$.

$$\hat{Z} = \{\widehat{T_1}, \widehat{T_2}, \ldots\ldots, \widehat{T_N}\}, \tag{3}$$

Now consider an unsupervised learning problem where the set of transaction sequences Z is taken as input after performing feature value separation, then anomaly detection refers to identifying whether the current user behavior is anomalous or not. It is based on whether the predicted value $\widehat{x_t}$ is significantly different from the unique feature vector $\widehat{T_t}$, and comparing the degree of difference with the threshold value yields the anomalous label. Thus, the training input contains only normal transaction sequences.

To further illustrate the anomaly detection strategy implemented in this paper, a sliding window $\mathcal{W}_t$ is now defined, i.e. there is a sequence of length L at time t of the current operation:

$$\mathcal{W}_t = \{W_{t-L+1}, W_{t-L+2}, \ldots\ldots, W_{t-1}, W_t\}, \tag{4}$$

The original normal behavior sequence Z can be transformed into a series of windows $\mathcal{W} = \{\mathcal{W}_1, \mathcal{W}_2, \ldots\ldots, \mathcal{W}_T\}$ as training input. After feeding them into the model, the model will output two vectors: $feature$、$predict$. Now given a binary $y \in \{0, 1\}$, the goal of the anomaly detection problem is to compute the anomaly score $distance$ based on the window prediction value $\widehat{x_t}$. By comparing the threshold value θ to determine whether the current operation is anomalous or not. If $distance > \theta$ then the point to be detected is anomalous behavior and $y_t = 1$, otherwise $y_t = 0$. The anomaly score is calculated as shown in equation (5):

$$distance\big(\quad \text{s},\ \widehat{T_N}\big) = \sqrt{\sum_{i=1}^{m}\big(x_t - \widehat{T_N}\big)^2}, \tag{5}$$

where x_t is the predicted value obtained by the model using $\{\widehat{T_1}, \widehat{T_2}, \ldots\ldots, \widehat{T_{N-1}}\}$ as an input to the LSTM, and $\widehat{T_N}$ is the feature vector of the Nth thing containing information about the n features.

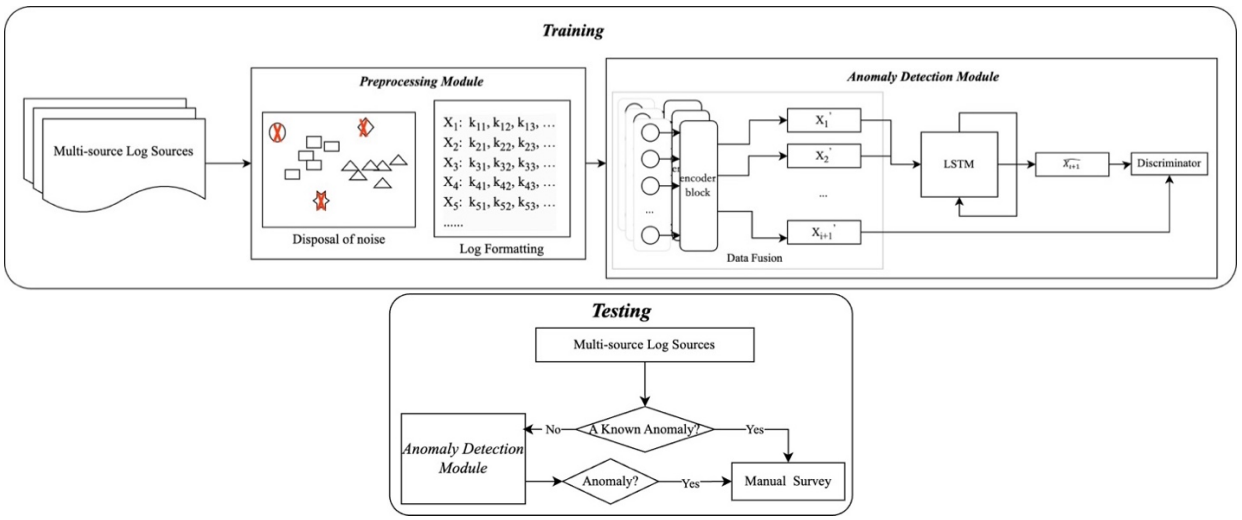


FIGURE 1. Overview of the anomaly detection method, which is divided into two phases: offline training and online detection. Where offline training consists of a preprocessing module and anomaly detection module.

B. The TE-LSTM Model

System logs are recorded according to time, but analyzing temporal data only according to its characteristics is not applicable as a basis for anomaly detection, so we expect to predict temporal data through multiple dimensions. In this paper, the prediction model is trained with a normal dataset to predict the evolution of the data, and the degree of discrepancy between the obtained predicted values and the actual observed values is calculated. This variance analysis is effective in identifying behaviors that covertly deviate from normal patterns, thus providing a solid basis for further monitoring and analysis.

In order to capture features in both the semantic and temporal dimensions of normal sequences, this paper proposes a model that combines a Transformer-Encoder and an LSTM, called TE-LSTM. The goal is to analyze the contextual semantics in normal sequences, extract the feature vectors, and use the sequences after dimensional compression as input to the LSTM to analyze the temporal correlations.

LSTM performs better when processing temporal data. However, in the application scenario of this paper, due to the high preprocessing requirements of LSTM on the input data and the expectation that the training data comprehensively covers all possible normal operating modes. Therefore, it is inaccurate to rely only on LSTM to learn the sequence of normal behaviors, otherwise, the LSTM cannot correctly identify the abnormal behaviors.

The encoder part of the traditional Transformer model captures the relevance of the input through the mechanism of

multiple attention. Positional coding is introduced to take into account the effect of the positional information of each word in the text message on the final result. However, the coder that introduces positional coding is not conducive to accurately extracting the feature vectors of the sequence. In addition, the traditional coder chooses a mask-free fully connected attention design without masks. The feature vectors extracted by fully connected attention are highly correlated with information about specific feature values. This results in feature vectors that contain a lot of repetitive information and do not describe the contextual features well.

In order to better preserve the contextual features, the method in this paper improves the traditional Transformer encoder. By removing the positional coding and introducing a masking mechanism, the model can extract key features in the sequence more accurately while reducing the repetition of redundant information. The key requirement for feature vectors is that they can comprehensively characterize the global pattern of the sequence and avoid losing long-range dependencies by over-emphasizing local associations.

Preserving contextual features can better characterize potential associations in the sequence, especially for anomaly detection scenarios where anomalies are often embedded in complex patterns in the context. Therefore accurately capturing these patterns is crucial for improving detection.

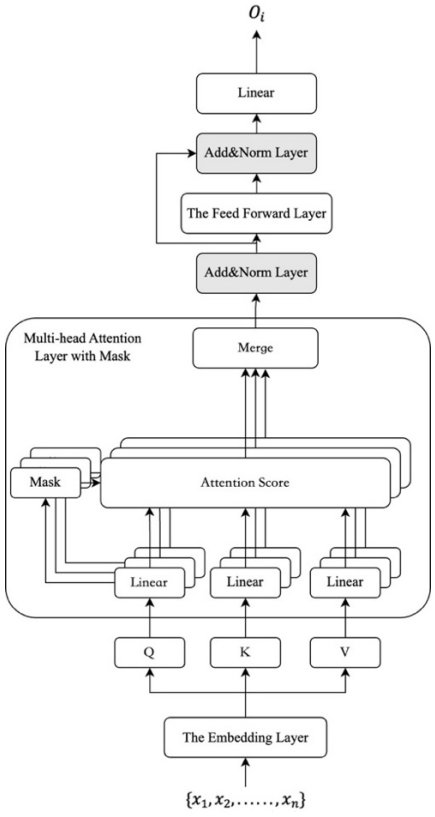


FIGURE 2. **The details of the composition of the encoder block.**

To solve the above problems, this paper combines the advantages of LSTM and Transformer and proposes an anomaly detection method based on the TE-LSTM model. As shown in Figure 2, The TE-LSTM model consists of several encoder blocks and LSTM modules, and each encoder block consists of an embedding layer, a multi-attention layer with masks, and a linear layer. The embedding layer is used to embed the feature values without relative position information in each sequence. The attention layer with masks connects the current operation with its bidirectional operation context, except for the operation itself. The masking mechanism prevents inferring the semantics of the operation directly from the operation itself. The length of the output sequence of this part is the same as that of the input sequence. After extracting the semantic features, the output of the encoder module will be used as input to the LSTM for analyzing the sequence. The output of LSTM is used for anomaly detection.

The Embedding Layer. The embedding layer maps each feature value token in the input sequence to its corresponding embedding vector. These word embedding vectors contain semantic information about the words and can be learned in the model. The embedding layer constructs an embedding matrix $M_e \in R^{n\times h}$, and for each word in the input sequence, the corresponding embedding vectors are looked up in the embedding matrix according to the index, and these embedding vectors form the vector representation of the input sequence. Now a sequence x of length N is taken as input, and the input sequence, after passing through the embedding layer, can be represented as:

$$E = x \cdot M_e = \{e_1, e_2, \ldots, e_N\}, e_i \in R^h \tag{6}$$

Unlike the conventional embedding layer, we remove the position encoder and ignore the effect of the relative position on the extracted features in the subsequent analysis.

Multihead Attention Mechanism Layer with Mask. The multiple attention mechanisms layer with the mask is based on a multi-head attention mechanism that extends the capabilities of the self-attention mechanism, where each head can learn and capture different features or patterns in the input sequence. The attention mechanism can be viewed as a function that maps a query and a set of key-value pairs to an output, where the query, keys, and values are vectors that are individually projected from the input data[15]. Based on the property that the self-attention mechanism only considers dependencies within the input sequence, the sequence of operations is transformed into a semantic representation. The output sequence $E \in R^{N\times h}$ of the embedding layer is used as input to the multi-head attention layer. Set m attention heads, and for each attention head there are:

$$\begin{aligned} Q_i &= EW_i^Q, \\ K_i &= EW_i^K, \\ V_i &= EW_i^V, \end{aligned} \tag{7}$$

where W_i^Q、W_i^K、$W_i^V \in R^{h\times h/m}$ are the weight matrixes for each attentional head. The output of the calculation of each head can be expressed as:

$$\begin{aligned} HD_i(E) &= Attention(Q_i, K_i, V_i), \\ Attenrtion &= softmax\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i, \end{aligned} \tag{8}$$

Where d_k denotes the dimension of the keyword vector. The formula is computed to obtain an $N \times N$ weight matrix representing the contextual relevance of the current sequence. The computation procedure is as in equation (8), where the dot product of each query with all keywords is computed and subsequently divided by the dimension of d_k.

In general, multiplying the weight matrix by the value matrix V_i can give the attention matrix. However, in this paper, we want the model to pay more attention to the front and back positions of the current sequence and ignore its features to reduce the redundant information in the feature vector. In the Transformer's decoder module, prediction is only affected by the future-masking mechanism, which uses only the unidirectional terms before the ith+1st input to predict the ith output. Inspired by this masking mechanism and to better achieve the goal of this paper, a new masking mechanism is designed. Specifically, the computation of Q_i and K_i is decoupled to eliminate the effect of the current operation on the final feature vector. Then equation (9) can be written as follows:

$$Attention = softmax\left(\frac{Q_i K_i^T}{\sqrt{d_k}} + M\right)V_i, \quad (9)$$

Where the matrix $M \in R^{N\times N}$ is a diagonal matrix and the values of the diagonal elements are positive infinity. The values of the diagonal elements after the $softmax$ operation are 0. Multiple heads of attention are then realized by multiple parallel layers of self-attention, which are now combined with m heads of attention, and here is equation (10):

$$MH(E) = Concat(HD_1(E), HD_2(E), \dots, HD_m(E))W^O, \quad (10)$$

Where $W^O \in R^{h\times h}$ is the matrix used to concatenate the outputs of multiple heads prior to a linear transformation. Then, in the next process, the obtained matrix is subjected to layer normalization (Eq. 11) and residual crosstabulation (Eq. 12). The aim is to alleviate the problem of vanishing gradients, stabilize the input distribution of the layers, speed up the training convergence, and improve the performance and stability of the model.

$$LN(x) = \frac{x-\mu}{\sqrt{\sigma+\epsilon}} \odot \gamma + \beta, \quad (11)$$

$$RC(x) = LN(x + Dropout\ (f\ (x))), \quad (12)$$

Where γ and β in equation (11) are the gain and bias parameters, which are trainable parameters. μ, σ are the mean, and the variance of x. $\odot$ is the element-by-element multiplication between two vectors. ϵ is a small constant to prevent division by zero. The $f\ (x)$ in equation (12) denotes the output of the multi-head attention layer, and the $Dropout$ is to mitigate the overfitting problem. The multiple attention mechanisms layer with the mask is $RC(x)$.

The Feed Forward Layer. Each attention mechanism layer is followed by a feedforward layer whose role is to linearly transform the input to capture richer features. It consists of two linear transformations and a non-linear activation function:

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2, \quad (13)$$

Where x is the input matrix, i.e. the output of the attention layer. $b_1, b_2 \in R^h$ is the bias vector. W_1, $W_2 \in R^{h\times h}$ are the weight matrixes and learnable parameters and are shared across all positions. The max function is then an activation function. Similarly, the layer is followed by a residual and normalization operation.

The Linear Layer. Finally, in order to extract the feature vectors, the obtained results are linearly transformed, and by learning the weight matrix and bias vector, the model can extract and combine features from the input data. The specific process is described below:

$$Linear(x) = FFN(x)W + b, \quad (14)$$

Where $W \in R^m$ is the weight matrix and b is a bias parameter. Therefore, the output feature vector of the ith encoder block can be expressed as:

$$O_i = Linear(x), i \in [1, N], \quad (15)$$

LSTM Layer. Recurrent Neural Networks (RNN) are a class of artificial neural networks with backward connectivity, where the output of a network layer is fed back to that or a previous network layer. RNNs can solve the problems of traditional feed-forward neural networks[12], while the LSTM algorithm is explicitly proposed as a solution to avoid the problem of gradient vanishing. It uses the mechanism of gates to regulate the flow of information. The LSTM is composed of three control gates consisting of: a forgetting gate, an output gate, and an input gate. The forgetting gate is responsible for retaining part of the information from the previous state. The output gate is responsible for choosing how much information to output. The input gate is responsible for acquiring new information.

The previously extracted sequence features are represented as $\{O_{t-L+1}, O_{t-L+2}, \dots\dots, O_t\}$, which are used as inputs for analyzing temporal features, where O_i is a vector of high-dimensional contextual feature representations, which is used as input to the LSTM layer to analyze the temporal correlation of the feature vectors. We perform a dimensionality reduction operation on O_i by linear transformation for subsequent LSTM analysis. To ensure time step alignment, we omit the data of the last time step as the LSTM input, i.e., $I = \{O_1, O_2, \dots, O_{N-1}\}$. By doing this the LSTM learns the long-term dependence of the historical time step and finally outputs the predicted value x_t, Finally, we compute its similarity with O_N by equation (5).

The calculated similarity d is compared with the threshold value θ, and if $the\ distance(x_t, O_N) \geq \theta$, then the current behavior is abnormal, and vice versa. As the sequences used during training are normal behavior sequences, the final θ value is determined by constantly adjusting the comparison.

IV. IMPLEMENTATION

As mentioned above, anomaly detection is implemented based on a trained TE-LSTM. The method is divided into three phases. The first phase is the data pre-processing phase. In order to make our method applicable to structured logs, we parsed the logs through Drain to analyze the log templates and parameters to form a formatted log. Next, a vocabulary list is constructed based on the structured logs. Segmentation is performed first and word frequency filtering and word

clustering are used to compress the size of the vocabulary, then the vocabulary is constructed and the calendar is encoded. A sliding window of length T is then set to divide a sequence of length L into several time windows. Algorithm 1 summarises the process of generating time windows.

Algorithm 1: Generation time windows

```
input: file name, window_size
output: dataset
num_sessions ← 0
inputs ← []
files ← open file name
for line in file lines:
    num_sessions += 1
    for i in (len(line) - window_size + 1):
        inputs.append(line[i:i+window_size+1]
    end for;
end for;
dataset ←TensorDataset(inputs)
return dataset
```

In Algorithm 1, the input file is first read, and each line of the data is processed sequentially. Assume the file contains N lines, with each line having a length of L. The sliding window method is used to divide each line of data into multiple time windows of size $window\_size + 1$, and these window data are stored in the inputs list. For each line in the file, the time complexity of processing that line is $O(L)$, so the total time complexity for iterating through all the lines is $O(N \times L)$. For each line, the sliding window operation will be performed $L - window\_size + 1$ times, and each operation has a time complexity of $O(window\_size)$, so the time complexity of the sliding window operation is $O(L \times window\_size)$.

The second phase is the training phase, which is performed offline to make the feature extraction and LSTM predictions increasingly similar and to calculate their anomaly scores. A series of time windows generated by Algorithm 1 are used as inputs to calculate the similarity between the encoder outputs and the LSTM outputs, thus training the weight matrix of the model. Algorithm 2 summarises the model training process.

Algorithm 2: TE-LSTM algorithm

```
input: dataset W = {W_1, W_2, ....., W_T}, epochs N
output: feature, predict
n←1
repeat
    for t = 1 to T do:
        EW_t ← E(W_t)
        MH_t ← MH(EW_t)
        O_t ← Linear(FFN(MH_t))
        for i=1 to t-1 do:
    P_{i+1} ← LSTM(O_i)
    feature ← O_t
    predict ← P_t
    loss ← MSE(O_t, P_t)
    end for;
n ← n + 1
until n=N
```

For each time step t, the computational complexity of input embedding is typically $O(T \times d)$, since an embedding operation must be performed for each input W_t. The computational complexity of multi-head attention is $O(T \times d^2)$, where d is the feature dimension. This is because multi-head attention involves queries, linear transformations of keys and values, and the computation of attention. The computation of attention involves the operation $T \times T$, so the complexity is $O(T^2 \times d)$, although the final complexity is $O(T^2 \times d)$ because multi-head attention has multiple heads. For each time step t, the processing is performed using an LSTM. The computational complexity of the LSTM is typically $O(T \times h)$, where h is the hidden layer dimension of the LSTM.

In each training round, a series of computations are performed for each time step t, so for each training round the computational time complexity is $O(T^2 \times d)$ (mainly determined by the computation of multiple attention and LSTM). There are N iterations per training round, so the total time complexity is $O(N \times T^2 \times d + N \times T \times h)$, where N is the number of training epochs, T is the length of the input sequence (number of time steps), d is the feature dimensions at each time step, and h is the dimension of the hidden layer of the LSTM. In most cases, $O(T^2 \times d)$ will dominate, especially when the sequence length T is long.

Algorithm 1 and Algorithm 2 show the implementation steps of the two phases of data pre-processing and training, respectively. First, Drain is used to parse the structured information of the logs, and the sliding window method (Algorithm 1) is used to divide the log data into time windows of fixed size, providing input data for the TE-LSTM model. Then, in the training phase (Algorithm 2), TE-LSTM extracts the features of each window and uses LSTM for time series prediction to optimize the loss function so that the two features match.

The final stage is anomaly detection. It uses the model trained in the second stage for online detection. When a new sequence arrives, the model is used to obtain an anomaly score. If the anomaly score of a window is higher than the defined anomaly threshold, the new time window is considered anomalous.

V. EVALUATION AND RESULTS

A. The TE-LSTM Model

HDFS log dataset. The dataset was generated by running Hadoop-based map-reduce jobs on over 200 EC2 nodes at Amazon and tagged by Hadoop domain experts. Of the 11,197,954 log entries collected, approximately 2.9% were anomalies, including events such as 'write anomaly'. This was the first large dataset used in an online PCA-based approach[17] and has been used in several subsequent works, including online PCA approaches[18] and IM-based approaches[19]. For more information on this dataset, see the related paper by Xu et al.[17].

BGL dataset[20]. This dataset is a collection of data collected by the University of California, Berkeley (UC Berkeley) to analyze and investigate system performance related to Garbage Collection (GC), especially for applications

in distributed computing environments. It was generated by the Blue Gene/L super-computer.

TE-LSTM does not need to use all HDFS logs for training, so we use less than 1% of these normal sessions (4,855 sessions parsed from the front-end 100,000 log entries, out of a total of 11,197,954 log entries) for training. For the HDFS and BGL dataset, according to Algorithm 1, there are 46575 and 1056541 sequences for training respectively, these sequences are different log samples extracted from the dataset, which play a vital role in verifying whether TE-LSTM can extract semantic features effectively. Table 1 summarises the basic structure of the dataset.

TABLE 1
OVERVIEW OF DATASET

Dataset	Data volume		Glossary size
	Training data	Test data	
HDFS	Normal data: 4855	Normal data: 553366 Abnormal data: 16838	29
BGL	Normal data: 831	Normal data: 5,990 Abnormal data: 453	656

B. Evaluation Metrics

In this paper, Precision, Recall, and F1-Score are used to evaluate the performance of the model. Precision represents the percentage of true anomalies among all detected anomalies. Recall is used to measure the percentage of anomalies detected in the dataset. F1-Score is the reconciled mean of the two. The mathematical formulas for these metrics are shown below:

$$Presicion = \frac{TP}{TP+FP}, \tag{16}$$

$$Recall = \frac{TP}{TP+FN}, \tag{17}$$

$$F1-Score = \frac{2*Presicon*Recall}{Presicon+Recall}, \tag{18}$$

Where TP is a true positive, FP is a false positive and FN is a false negative. Specifically, we consider that if a window contains an anomaly, then the user action corresponding to that window is marked as an anomaly.

C. Performance Metrics

We use the generated features $\hat{Z}$ to train our model so that the reconstruction loss is minimized. Several experiments were carried out to find the best values of hyperparameters for model initialization. In these experiments, the values of learning rate, hidden layer size, number of iterations, and batch size were varied to obtain higher accuracy. We finally train the model using the Adam optimizer with a learning rate of 0.0001, a batch size of 1024, and the Tanh activation function. We train the model for 100 epochs. Table 2 summarises the different hyperparameter choices.

TABLE 2
HYPERPARAMETER VALUES ARE SET IN THIS PAPER.

Parameters	Best Values
Hidden layers	2
hidden_size	48
Batch size	1024
Optimizer	Adam
function	MSE
Activation function	Tanh
Learning rate	0.0001
Number of epochs	100

We use normal HDFS data to train the model. The error between the extracted raw features and the output features is calculated. The $\ell 2 - norm$ error will be low for normal data, while the $\ell 2 - norm$ error will be high for abnormal data. Therefore, we use a fixed threshold in the error for binary classification of normal and abnormal log data, which is used as a decision boundary for detecting abnormal data. Figure 3 shows the performance of the model under different thresholds. If the error between the extracted original features and the output features is greater than the threshold, it is classified as abnormal data. Conversely, if the error is less than the threshold, it is classified as normal data.

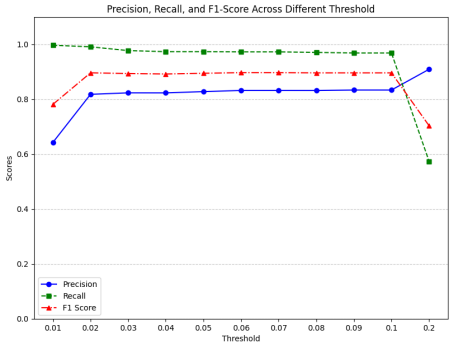


FIGURE 3. Evaluate metrics for different thresholds.

As shown in Figure 3, precision is lower when the threshold is relatively low, indicating more misjudgments in the predicted normal samples. As the threshold increases, the model becomes stricter in defining anomalies, resulting in a continuous improvement in Precision. At the same time, the model becomes more conservative, leading to more misclassifications of abnormal samples. The figure also shows that when the threshold reaches a certain value, the model indicators stabilize, suggesting that within this range, changes in the threshold have little effect on the model's decision boundary. This also validates the effectiveness of our method in differentiating normal from abnormal samples.

The above results also indicate that when selecting a reasonable threshold, multiple factors need to be considered to ensure that the model performs optimally in practical applications. Firstly, the model's evaluation metrics (such as accuracy, precision, recall, and F1 score) should be balanced. Therefore, when choosing the threshold, we should not only focus on improving accuracy but also consider the balance between precision and recall. As seen in the experimental results in Figure 3, as the threshold increases, the model's metrics gradually stabilize, indicating that within a certain range, the change in threshold has a diminishing effect on the model's decision boundary. Therefore, we can select a value within this threshold range as a reasonable threshold to avoid

over-influencing the model with excessively high or low thresholds.

D. Characteristic importance analysis

In anomaly detection tasks, feature importance analysis can help us understand how much the model relies on different features and the contribution of each feature in the model decision-making process. Through feature importance analysis, we can better interpret the model's predictions, identify potential critical factors, and provide model interpretability to enhance its credibility in real-world applications. In this study, we used two popular model interpretation tools, SHAP (Shapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) to analyze the features.

In our experiments, we chose the KernelExplainer interpreter for the global analysis of the model. KernelExplainer is suitable for any model and is especially suitable for black-box models that cannot be interpreted by direct computation of the gradient. The TE-LSTM model is a combinatorial architecture that involves several modules, including an Encoder, an LSTM module, and a number of linear layers, whose structure and decision-making process are difficult for the user to understand directly and is therefore considered to be a black-box model. KernelExplainer is a model-independent interpreter for any black-box model that provides an understanding of the model's decision-making process without access to the model's internal structure.

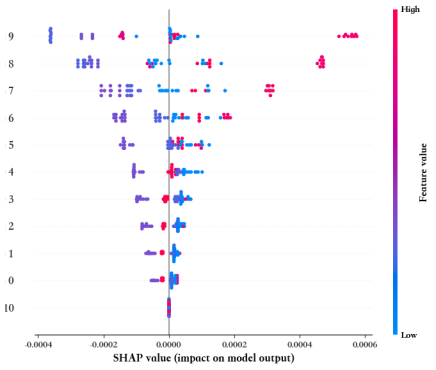
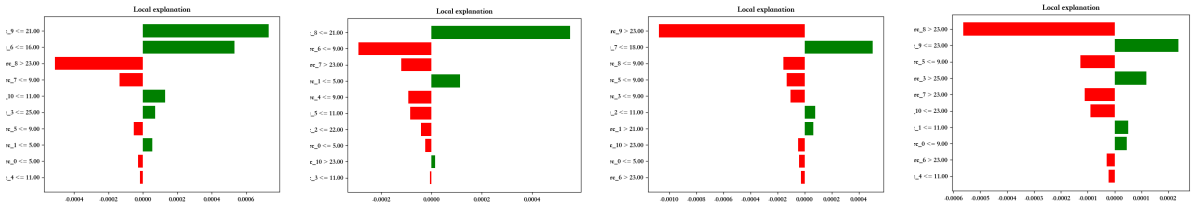


FIGURE 4. SHAP values of the items in the sequence and the model outputs

In the actual experiments, we set the window size to 11, and each feature is numbered from 0 to 10, where the feature value numbered 10 is used as the prediction contrast value. Specifically, features 0 to 9 represent the historical information of the input series, while feature 10 is the difference or contrast value between the model's prediction result and the true value. Figure 4 clearly demonstrates the performance of our model in processing time series, which can effectively capture complex patterns and trends in time-series data. Features 0 to 9 are presented sequentially about the predicted values, reflecting the predictive ability of the model at different time steps. As the time step progresses, the model demonstrates strong pattern recognition ability in processing the information in the sequence and can accurately capture potential anomalies or trend changes in the sequence.



(a) (b) (c) (d)

FIGURE 5. Local interpretation and character contribution analysis

LIME is a local interpretation method that approximates the decision boundary of a complex model by means of a local linear model. As shown in Figure 5, the horizontal coordinates indicate the importance of features or their contribution to the prediction results, and the vertical coordinates indicate the values of different features in the model. Each bar indicates the effect of a feature on the model prediction. The red bars indicate the negative impact of the feature on the model prediction and the green bars indicate the positive impact of the feature on the model prediction. We selected the 15th, 16th, 17th, and 18th samples for our analysis, and there is some variation in the impact of features on model predictions across the samples, with the influence not always following a time series. This confirms that our model does not only consider the time series, as in Fig. 5(d), the eigenvalues (numbered 0, 1, 3) that are far away from the predicted point in the series also have a positive influence on the predicted values. Although they are further away from the current prediction point in the time series, they may carry semantic information that is closely related to the prediction task. These features may represent some underlying patterns or semantic relationships that positively influence the prediction results. This phenomenon reflects that our model is not limited to the input data at the current moment in time series data processing, but can capture global information or long-term dependencies.

E. Overall performance

To demonstrate the overall performance of TE-LSTM, we compare it with three unsupervised methods for time series anomalies. These methods are Deeplog[21]、USAD[22]、UCAD[23]. Since not all anomaly detection methods used for comparison provide a mechanism for selecting anomaly thresholds, we tested the possible anomaly thresholds for each model and reported the results associated with the highest F1 scores. Table 3 shows the performance results of all methods on the HDFS dataset. It can be seen that our methods have improved in terms of precision, recall and F1 score.

On the HDFS dataset, The method proposed in this paper improves accuracy by 0.023 compared to the best-performing Deeplog method, with both recall and F1 scores showing relatively significant results. The recall indicates good performance in identifying anomalous sequences. The performance of UCAD and USAD is relatively lower because these two unsupervised anomaly detection methods do not utilize the temporal information between observations. Although USAD includes temporal relationships, it does not effectively extract temporal correlations between sequences. For time series data, temporal information is crucial; in this paper, we further process the temporal connections after feature extraction using LSTM. Consequently, the method proposed in this paper outperforms other methods in processing the HDFS dataset.

On the BGL dataset, the method in this paper improves both Precision and F1-score compared to the best performing UCAD method because both of them consider the contextual semantics of the log data, while Deeplog and USAD focus more on the temporal information of the logs, and thus perform slightly worse on the BGL dataset which has a higher amount of semantic information.

TABLE 3
COMPARISON OF EVALUATION METRICS. PRECISION, RECALL AND F1-SCORE VALUES ARE REPORTED FOR THE DIFFERENT 4 METHODS.

DataSet	Methods	Deeplog	USAD	UCAD	TE-LSTM
HDFS	Precision	0.8110	0.7945	0.7751	**0.8340**
	Recall	0.9199	0.8692	0.9181	**0.9689**
	F1-score	0.8621	0.8503	0.8406	**0.8964**
BGL	Precision	0.8974	0.7697	0.9045	**0.9104**
	Recall	0.8278	**0.9831**	0.9583	0.9778
	F1-score	0.8612	0.8186	0.9306	**0.9314**

By comparing the performance of HDFS and BGL datasets, it can be seen that the TE-LSTM model can combine the semantic information and temporal features of log data well. Especially when dealing with log data involving temporal associations, the LSTM model effectively captures the temporal dependencies without neglecting the features at the semantic level. This advantage enables TE-LSTM to perform well on a wide range of different log datasets, with stronger generalization ability and higher detection accuracy compared to traditional methods.

During the experiment, anomalies tend to be concentrated in certain time periods, especially when the system is experiencing problems. This increases the risk of false positives to some extent. Therefore, by using a sliding window approach, where multiple log entries within a given time range are treated as a whole when assessing anomalies, this method helps to reduce isolated false positives. Even if a single log entry appears anomalous in isolation, it may be considered a normal event if no other anomalies are observed in adjacent time windows.

F. Effect of parameters

In this section, we investigate the effects of different parameters and factors on the performance of TE-LSTM. It is worth noting that by adjusting the parameters, the threshold value will change, and from the previous section, the model performance will be stabilized in a certain threshold range because the normal and abnormal sequence $\ell 2 - norm$ errors will be concentrated in a certain range respectively. To make the results of different parameters fair, we choose the threshold value to stabilize the model performance range and use this result for comparison.

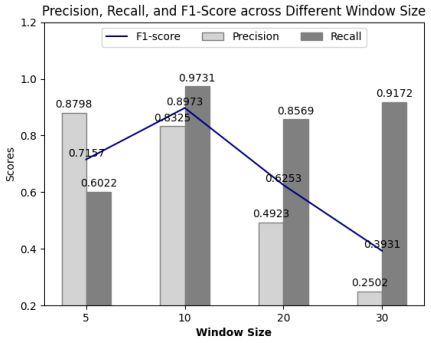


FIGURE 6. Effect of window size on model performance.

The first factor we investigate is how our method responds to different window sizes in the data. Window size affects the type of anomalous behavior that can be detected and directly affects the speed of anomaly detection. Due to the limitations imposed by the size of the HDFS dataset, our performance was tested on three different window sizes $L \in [5, 10, 20]$. Figure 5 shows the results for different window sizes and we can see that the best results are obtained with a window size of $L = 10$. When the window is small, the model learns the normal data features poorly and cannot separate the normal points from the anomalies. Although the accuracy is high, the recall value is very low, which means that the model is not sensitive to anomalous data. If the window value is too large, we can see that the accuracy rate is low, indicating that the model is learning too many features when determining whether the current session is abnormal or not, making the model

less accurate. For the HDFS dataset, the optimal window size is set at 10. Because the training speed and detection speed perform best when the window size is set to 10.

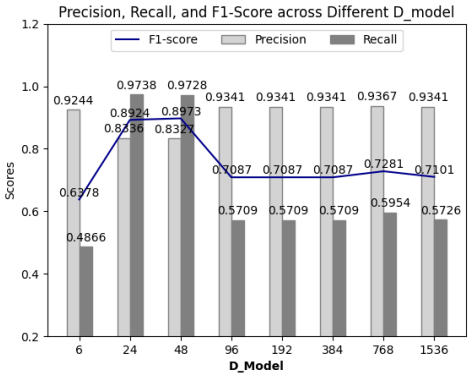


FIGURE 7. The effect of the dimensionality of the latent variable Z on model performance.

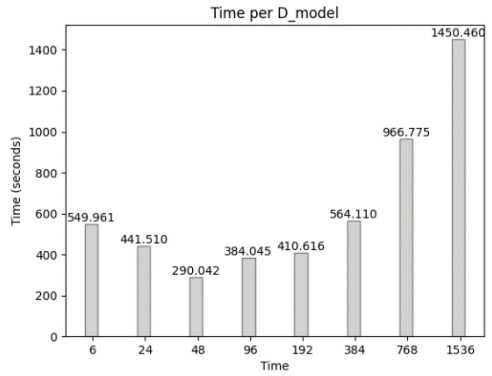


FIGURE 8. The effect of the dimensionality of the latent variable Z on the time required for model prediction.

The latent variable Z represents the output of the feature extractor and is an intermediate variable. We assume that the latent variable Z lies in m-dimensional space. Since the number of heads of the multi-head attention mechanism is set to 6, we investigate the effect of $m \in [6, 24, 48, 96, 192, 384, 768, 1536]$ on the performance of TE-LSTM. Figure 7 shows the results for different m dimensions, and Figure 8 shows the time spent for each m. The results show that when the dimension of Z is small, the model suffers from high accuracy and a high false alarm rate. This means that a lot of information is lost in the encoding phase and the model cannot distinguish between normal and abnormal data, leading to low performance. If we use a larger value of m, the model will have the same problem, because a larger value of m will lead to memory and performance degradation of the training data. In addition, we know from Figure 6 that when the value of m is large, the time taken by the model to make a prediction increases rapidly. Combining Figures 7 and 8, intermediate values of m have little effect on performance, showing a relatively high and stable F1-score.

G. Ablation Study

In the model design, we not only focus on the overall effectiveness of the architecture but also conduct an in-depth analysis of the contribution of semantic features to anomaly detection. To better understand the impact of semantic features on detection performance, we conducted multiple ablation experiments. When the model was trained using only LSTM to capture temporal features, the accuracy, recall, and F1 scores were all lower than those of TE-LSTM. This indicates that relying solely on temporal features is insufficient, and semantic features play a crucial role in anomaly detection. Figure 9 shows a comparison of the performance of the LSTM trained alone and with removing the masking mechanism. We can clearly show the change in the F1 score, and our model performs best without removing the training phase.

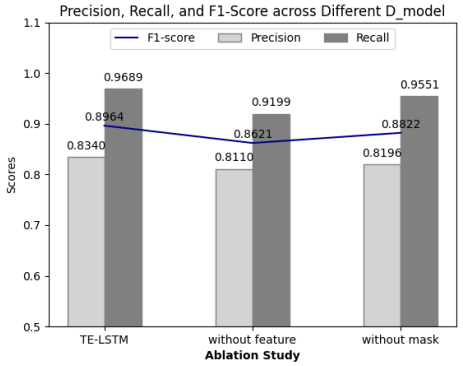


FIGURE 9. Adjustment of the effect of model details on the final result.

Moreover, the model employs a masking mechanism when extracting feature vectors to prevent excessive focus on individual feature values, thereby enhancing its generalization ability. The masking mechanism helps reduce the impact of low-value or redundant information, allowing the model to concentrate more on key features in log sequences and avoid performance degradation due to overfitting specific patterns. We conducted comparative experiments, and the results showed that removing the masking mechanism led to a decline in detection performance without significantly improving time efficiency. This indicates that the masking mechanism not only facilitates the extraction of anomalous patterns but also enhances model stability. Specifically, it suppresses certain high-frequency but non-contributory features, ensuring that the model does not overly rely on local information but instead fully integrates global semantics for anomaly detection.

The above experimental results further confirm the critical role of semantic features in anomaly detection. Time-based analysis alone is insufficient to fully capture the logical relationships between log entries, whereas the incorporation of semantic information helps the model more accurately identify anomalous patterns. Therefore, the design of the masking mechanism in TE-LSTM plays a

crucial role in enhancing the robustness and accuracy of anomaly detection.

H. Training time

In this section, we investigate the computational performance of TE-LSTM. Since the performance of the Deeplog method is the closest to the method in this paper, it is compared with it. The experiments are performed on machines with the same configuration, and the results obtained are listed in Table 4. Although the training time of TE-LSTM is longer than that of Deeplog, the method of this paper has a good performance in the testing phase.

TE-LSTM requires more time for training, but it performs better in testing because it excels at capturing complex patterns in the data during training. This allows it to make more accurate predictions during the test phase. The trade-off in training time is justified by the model's superior accuracy and robustness in detecting hidden anomalies in multi-source logs.

TABLE 4
TRAINING TIME PER METHOD PER EPOCH DURING TRAINING AND TIME USED FOR TESTING.

Method name	Training Time(s)	Testing Time(s)
Deeplog	9.78	349.35
TE-LSTM	15.95	290.04

The above experimental results show that our proposed TE-LSTM-based multi-source log anomaly detection method performs well in identifying hidden anomalies. The experimental results show that TE-LSTM outperforms other existing anomaly detection methods in terms of average accuracy, check-all rate and F1 score on HDFS datasets. The main reason for these results is that TE-LSTM combines the respective advantages of the two neural networks. On the one hand, allows the model to focus on the semantic information as well as the temporal information of the logs. The attention mechanism enhances the model's ability to capture critical information, thus improving the quality of data representation. On the other hand, TE-LSTM employs an appropriate regularisation strategy during the training process, which helps to reduce the risk of overfitting and improves the model's ability to generalize to unknown data. In summary, the design concept and implementation details of TE-LSTM together contribute to its excellent performance in the task of multi-source log anomaly detection.

Trans-Encoder and LSTM serve as the two main modules in the TE-LSTM model and are the main source of time overhead, as shown by the time complexity of Algorithm 2 in Section 4. In the traditional self-attentive mechanism, the query at each position must be compared with the keys at all other positions, which means that the computational complexity is the square of the length of the sequence, i.e. $O(T^2 \times d)$, where T is the length of the sequence. The computational complexity increases rapidly as the sequence length increases.

Sparse attention can effectively reduce the computational complexity of the attention mechanism, mainly by reducing the amount of computation between each query and all key values. The core idea of sparse attention is to consider only the attention relationship between a query and a small set of keys, rather than computing the attention of all key pairs. By limiting the scope of computation for each query, sparse attention can effectively reduce the amount of computation, especially when the sequence is long. In Reformer[25], hashed attention maps queries and keys to hash buckets using a hash function, and then computes attention only between items within the same hash bucket. The hashing process avoids global computation and thus reduces the amount of computation. The number of hash buckets is usually much smaller than the length of the sequence, which significantly reduces the amount of computation. The computational complexity of this method is $O(n \log n)$ rather than the traditional $O(n^2)$.

VI. CONCLUSION

In this paper, we present a hidden anomaly detection method based on the TE-LSTM model, which is a deep neural network-based approach for detecting hidden anomalies. The advantage of TE-LSTM is that it not only extracts the semantic information from the logs but also preserves the temporal features. The log parsing process is generic and does not impose special requirements on the format of the logs being analyzed, making our method's framework suitable for more comprehensive anomaly detection. It performs anomaly detection at the per-log-entry level, rather than being limited to the per-session level as many previous approaches have done. Experiments demonstrate that this method exhibits excellent performance in detecting hidden anomalies.

In addition to HDFS datasets, models can be used in many other areas. For example, in the Internet of Things (IoT) log analysis, models can improve the reliability and security of devices through anomaly detection, failure prediction, and security monitoring. In financial fraud detection, models can help financial institutions prevent fraud and reduce risk by analyzing transaction data, detecting anomalous transactions, and combating money laundering. In healthcare, models can be used for disease prediction and diagnosis, treatment effectiveness assessment and medical anomaly detection to support personalized health management. In addition, models have applications in smart manufacturing, enabling real-time monitoring of equipment health, optimizing production processes and improving product quality. In social media and cybersecurity, models can analyze social data, identify malicious behavior and false information, and improve platform security. Finally, in intelligent transport and city management, models can predict traffic flows, optimize

traffic signals, detect accidents and provide early warnings to ensure transport safety and efficiency. In short, the application of these models in various industries is helping to improve automation, optimize the decision-making process and provide accurate solutions to real-world problems.

For future work, we plan to conduct more extensive experiments to investigate the applicability of TE-LSTM in various domains, such as IoT logs, financial fraud detection, and healthcare, where anomaly detection plays a crucial role. We also aim to explore additional techniques for enhancing model efficiency, such as pruning, quantization, and transfer learning, which can help make the model more feasible for real-time and resource-constrained environments.

REFERENCES

[1] '2023-data-breach-investigations-report-dbir.pdf'. Accessed: Apr. 02, 2024. [Online]. Available: https://www.verizon.com/business/resources/T2b1/reports/2023-data-breach-investigations-report-dbir.pdf

[2] '(PDF) New insider threat detection method based on recurrent neural networks', *ResearchGate*, Oct. 2024, doi: 10.11591/ijeecs.v17.i3.pp1474-1479.

[3] L. Liu, O. De Vel, C. Chen, J. Zhang, and Y. Xiang, 'Anomaly-Based Insider Threat Detection Using Deep Autoencoders', in *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*, Singapore, Singapore: IEEE, Nov. 2018, pp. 39–48. doi: 10.1109/ICDMW.2018.00014.

[4] L. Zhang, Y. Xie, L. Xidao, and X. Zhang, 'Multi-source heterogeneous data fusion', in *2018 International Conference on Artificial Intelligence and Big Data (ICAIBD)*, May 2018, pp. 47–51.

[5] H. Eldardiry, E. Bart, J. Liu, J. Hanley, B. Price, and O. Brdiczka, 'Multi-Domain Information Fusion for Insider Threat Detection', in *2013 IEEE Security and Privacy Workshops*, May 2013, pp. 45–51.

[6] D. Zhang, T. Wu, X. Zhou, B. Hu, and W. Zhang, 'Multi-source System Log Behavior Pattern Mining Method Based on FP-Growth', in *Proceedings of the 2023 International Conference on Communication Network and Machine Learning*, Zhengzhou China: ACM, Oct. 2023, pp. 248–254.

[7] S. Huang *et al.*, 'HitAnomaly: Hierarchical Transformers for Anomaly Detection in System Log', *IEEE Trans. Netw. Serv. Manage.*, vol. 17, no. 4, Dec. 2020, pp. 2064–2076.

[8] F. Yuan, Y. Shang, Y. Liu, Y. Cao, and J. Tan, 'Attention-Based LSTM for Insider Threat Detection', in *Applications and Techniques in Information Security*, V. S. Shankar Sriram, V. Subramaniyaswamy, N. Sasikaladevi, L. Zhang, L. Batten, and G. Li, Eds., Singapore: Springer, 2019, pp.

[9] S. Lee, S.-J. Park, and K.-S. Hong, 'RDFNet: RGB-D Multi-level Residual Feature Fusion for Indoor Semantic Segmentation', in *2017 IEEE International Conference on Computer Vision (ICCV)*, Oct. 2017, pp. 4990–4999.

[10] P. Bilinski and V. Prisacariu, 'Dense Decoder Shortcut Connections for Single-Pass Semantic Segmentation', in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Jun. 2018, pp. 6596–6605.

[11] V.-H. Le and H. Zhang, 'Log-based Anomaly Detection Without Log Parsing', in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Melbourne, Australia: IEEE, Nov. 2021, pp. 492–504.

[12] S. Liu, L. Xu, J. Wang, Y. Sun, and Z. Qin, 'LCR-GAN: Learning Crucial Representation for Anomaly Detection', in *2021 5th International Conference on Computer Science and Artificial Intelligence*, Beijing China: ACM, Dec. 2021, pp. 162–167.

[13] S. Mao, J. Guo, T. Gu, and Z. Ma, ..'Dis-AE-LSTM: Generative Adversarial Networks for Anomaly Detection of Time Series Data', in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, Oct. 2020, pp. 330–336.

[14] M. Said Elsayed, N.-A. Le-Khac, S. Dev, and A. D. Jurcut, 'Network Anomaly Detection Using LSTM Based Autoencoder', in *Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks*, Alicante Spain: ACM, Nov. 2020, pp. 37–45.

[15] A. Mohammed, M. Kashif, M. H. Zama, M. A. Ansari, and S. Ali, 'Master GAN: Multiple Attention is all you Need: A Multiple Attention Guided Super Resolution Network for Dems', in *IGARSS 2023 - 2023 IEEE International Geoscience and Remote Sensing Symposium*, Jul. 2023, pp. 5154–5157.

[16] C. Yin, Y. Zhu, J. Fei, and X. He, 'A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks', *IEEE Access*, vol. 5, pp. 21954–21961, 2017.

[17] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, 'Detecting large-scale system problems by mining console logs', in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, Big Sky Montana USA: ACM, Oct. 2009, pp. 117–132.

[18] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, 'Online System Problem Detection by Mining Patterns of Console Logs', in *2009 Ninth IEEE International Conference on Data Mining*, Dec. 2009, pp. 588–597.

[19] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, 'Mining Invariants from Console Logs for System Problem Detection'.

[20] A. Oliner and J. Stearley, 'What Supercomputers Say: A Study of Five System Logs', in *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, Edinburgh, UK: IEEE, Jun. 2007, pp. 575–584.

[21] M. Du, F. Li, G. Zheng, and V. Srikumar, 'DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning', in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, Dallas Texas USA: ACM, Oct. 2017, pp. 1285–1298.

[22] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, 'USAD: UnSupervised Anomaly Detection on Multivariate Time Series', in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Virtual Event CA USA: ACM, Aug. 2020, pp. 3395–3404.

[23] S. Li, Q. Yin, G. Li, Q. Li, Z. Liu, and J. Zhu, 'Unsupervised Contextual Anomaly Detection for Database Systems', in *Proceedings*

[24] *of the 2022 International Conference on Management of Data*, Philadelphia PA USA: ACM, Jun. 2022, pp. 788–802.

[25] N. Kitaev, Ł. Kaiser, and A. Levskaya, 'Reformer: The Efficient Transformer', Feb. 18, 2020, *arXiv*: arXiv:2001.04451.

Chen Luo was born in Nanchang, Jiangxi, China, in 1987. He received the B.E. degree in engineering from Nanchang University, Nanchang, China, in 2008, and the Ph.D. degree in engineering from Soochow University, Suzhou, China, in 2018. His primary research interests include data security, machine learning, urban computing, and recommender systems.

He worked as an Algorithm Specialist with Bilibili Technology Co., Ltd., Shanghai, China, from 2018 to 2022. Since 2022, he has been a Research and Development Engineer with the Applied and Data Security Technology Research Division, China Electric Power Research Institute, Beijing, China. He has published four research papers and holds two patents. His current and previous research focuses on advancing data security technologies and their applications in intelligent systems.

Dr. Luo is a member of the Jiangsu Cyberspace Security Association and serves as a committee member of the AI Security Special Committee.

Xinzhe Wang was born in Weifang, Shandong, China, in 1997. He received the B.E. degree from Harbin Institute of Technology, Harbin, China, in 2020, and the M.S. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2021. His primary research interests include intelligent power grids and advanced engineering applications.

He joined the State Grid Smart Grid Research Institute Co., Ltd., Beijing, China, as an R&D Engineer in 2023. In 2024, he began working as a Business Engineer with the China Electric Power Research Institute, Beijing, China. Since joining, he has co-authored one research paper and holds two patents. His work focuses on the intersection of power grid intelligence and engineering innovations.

Mr. Wang is actively involved in collaborative research and innovation, with a focus on advancing the field of intelligent power systems.

Zhou Peng was born in Shengzhou, Zhejiang, China, in June 1976. Zhou Peng holds a bachelor's degree and is a Senior Engineer at the State Grid Corporation of China, an expert in information systems. His research interests include power system digitalization, smart grid security and optimization, and big data analytics in the power sector.

Starting in 1998, he has been engaged in the development of power information systems, focusing on the digitalization and maintenance of power system infrastructure. Since 2006, he has been working at State Grid Zhejiang Electric Power Company, Hangzhou, China, overseeing the planning, construction, and operation of centralized information systems.

Mr.Zhou has authored four research papers and holds five invention patents. Zhou is dedicated to advancing power system digitalization technologies and enhancing the intelligence and security of the power grid. He has led and participated in numerous provincial and ministerial-level scientific research projects, receiving three Science and Technology Progress Awards at the provincial and ministerial levels.

Zhipeng Shao was born in Nanjing, Jiangsu, China, in May 1984. He received the B.E. degree in engineering from Zhejiang University, Hangzhou, China, in 2006, and the M.E. degree in engineering from Nanjing University, Nanjing, China, in 2014. His primary research interests include power system cybersecurity frameworks and critical technology development.

He is a Senior Engineer and currently serves as the Director of the Applied and Data Security Technology Research Division at the China Electric Power Research Institute, Beijing, China. He is also a member of the Digital Technology Standards Professional Working Group of the State Grid Corporation of China and a core member of the National Grid Cybersecurity Dynamic Defense Research Team. Over the years, he has been deeply involved in research on cybersecurity protection frameworks and key technologies for power systems. He has contributed to industry-level five-year network security plans, security protection for power monitoring systems, and secure design for smart grids.

Mr. Shao has led and participated in several major national and State Grid Corporation scientific research projects. He has received six provincial and ministerial-level science and technology awards and holds more than ten authorized invention patents.

Wangshu Sun was born in Wenzhou, Zhejiang, China, in 1991. She received the B.E. degree in engineering from Beijing Forestry University, Beijing, China, in 2014, and the M.M. degree in management from Northwestern Polytechnical University, Xi'an, China, in 2024. Her research interests include networks, information security, and system operations.

From September 2014 to August 2021, she worked as a Specialist with the State Grid Wenzhou Power Supply Company, Wenzhou, China. From September 2021 to August 2023, she served in a similar role at the State Grid Zhejiang Information and Communication Company, Hangzhou, China. Since September 2023, she has been a Specialist with the Digitalization Work Department at State Grid Zhejiang Electric Power Company, Hangzhou, China.

Ms. Sun has received several prestigious awards, including the Team Grand Prize in the National Cybersecurity Management Vocational Skills Competition for the Telecommunications and Internet Industry, recognition for Technological Innovation and Application at the China Energy Cybersecurity Conference, and First Prize in the Special Competition of the Power Industry Digital Intelligence New Technology Application Innovation Competition. She has authored four research papers and holds five invention patent.

Hongcheng Jiang was born in Shengzhou, Zhejiang, China, in June 1974. He holds a bachelor's degree and is a Senior Engineer. He currently serves as a Chief Expert at the State Grid Corporation of China.

Since 1996, he has been engaged in the development of power marketing systems. Since 2006, he has been working with the State Grid Zhejiang Electric Power Company, Hangzhou, China, focusing on the construction and operation of provincially centralized information systems.

Mr. Jiang has received five provincial and ministerial-level Science and Technology Progress Awards for his contributions to the field. His work has played a significant role in advancing the digital transformation and reliable operation of power systems.